PyTorch and tensors

1. Reading data

Data:

Feet	Inches	Meter
…	…	…
75	9	23.09
42	2	12.85
99	8	30.38
50	11	15.52
55	5	16.89
67	10	20.68
95	10	29.21
31	1	9.47
22	5	6.83
65	6	19.96
95	2	29.01

Read data:

```
print("NumPy array:")
data = np.loadtxt("feet_inches_meters.tsv")
print(data)
```

```
NumPy array:
[[63.    7.   19.38]
 [79.   10.   24.33]
 [65.    2.   19.86]
 ...
 [22.    5.    6.83]
 [65.    6.   19.96]
 [95.    2.   29.01]]
```

Read data:

```
1 print("NumPy array:")
2 data = np.loadtxt("feet_inches_meters.tsv")
3
4 print("Type:", type(data))
5 print("Shape:", data.shape)
6 last_el = data[999, 2]
7 print("'Last' element:", last_el)
8 print("'Last' element type:", type(last_el))
```

```
NumPy array:
Type: <class 'numpy.ndarray'>
Shape: (1000, 3)
'Last' element: 29.01
'Last' element type: <class 'numpy.float64'>
```

Create data in tensor-format

```
1 data = np.loadtxt("feet_inches_meters.tsv")
2
3 print("\nData tensors:")
4 data = torch.tensor(data.astype(np.float32), device='cpu')
5 print(data)
```

```
Data tensors:
tensor([[63.0000,  7.0000, 19.3800],
        [79.0000, 10.0000, 24.3300],
        [65.0000,  2.0000, 19.8600],
        ...,
        [22.0000,  5.0000,  6.8300],
        [65.0000,  6.0000, 19.9600],
        [95.0000,  2.0000, 29.0100]])
```

Divide each sample into input data, and the value we want the model to predict, given that input data.

```
data = np.loadtxt("feet_inches_meters.tsv")

data = torch.tensor(data.astype(np.float32), device='cpu')
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(3):
    for nr, example in enumerate(data):
        x, y = example[:2], example[2:]
        print("x_training:", x)
        print("y_expected:", y.item())
```

```
x_training: tensor([63.,  7.])
y_expected: 19.3799991607666
```

A linear model without bias is about as simple as you can get: Try to find w_1 and w_2, so that for all **x** (feet and inches) in you training data, predict the expected y (meter).

$w_1 * x_1 + w_2 * x_2$

x_1 = feet

x_2 = inches

```
1
2  model = torch.nn.Linear(2, 1, bias=False)
3  optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)
4
5  for epoch in range(3):
6      for nr, example in enumerate(data):
7          x, y = example[:2], example[2:]
8          print("x_training:", x)
9          print("y_expected:", y.item())
```

---

2. Training a model

Choose model and algorithm for optimising the parameters

```
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)
```

The five steps for training a model in pyTorch:

1. Make a reset of all parameter gradients
2. Make a prediction using current model parameters and the features of the training data
3. Compute loss (based on the prediction and the expected value in the training data)
4. Compute parameter gradients from the loss
5. Update parameters, given gradients and learning rate

```
for epoch in range(3):
    for nr, example in enumerate(data):
        w = model.weight.detach().cpu().numpy()[0]
        print(f"\nw = [{w[0]:8.4f}, {w[1]:8.4f}]")

        x, y = example[:2], example[2:]
        print("x_training:", x)
        print("y_expected:", y.item())

        model.zero_grad() # 1. Make a reset
        y_pred = model(x) # 2. Predict
        loss = (y_pred - y)**2 # 3. Compute loss
        print("y_predicted:", y_pred.item())
        print("loss:", loss.item())

        loss.backward() # 4. Compute gradients from loss
        optimizer.step() # 5. Update weights
```

```
    for nr, example in enumerate(data):
        x, y = example[:2], example[2:]
        model.zero_grad() # 1. Make a reset
        y_pred = model(x) # 2. Predict
        loss = (y_pred - y)**2 # 3. Compute loss
        loss.backward() # 4. Compute gradients from loss
        optimizer.step() # 5. Update weights
```

```
w = [ -0.3761,  -0.3137]  x_training: tensor([63.,  7.])
y_expected: 19.3799991607666
y_predicted: -25.892093658447266
loss: 2049.5625

w = [  0.1943,  -0.2503]  x_training: tensor([79., 10.])
y_expected: 24.329999923706055
y_predicted: 12.846368789672852
loss: 131.873779296875

w = [  0.3757,  -0.2274]  x_training: tensor([65.,  2.])
y_expected: 19.860000610351562
y_predicted: 23.968414306640625
loss: 16.87906265258789
```

3. Creating tensors

with and without `requires_grad`

Create data in tensor-format

```
data = np.loadtxt("feet_inches_meters.tsv")
data = torch.tensor(data.astype(np.float32), device='cpu')

print(type(data))
print(data.shape)
print("Requires grad?", data.requires_grad)

last_el = data[999, 2]
print("\n'Last' element:", last_el)
print("'Last' element type:", type(last_el))
print("'Last' element grad:", type(last_el.grad))
```

```
<class 'torch.Tensor'>
torch.Size([1000, 3])
Requires grad? False

'Last' element: tensor(29.0100)
'Last' element type: <class 'torch.Tensor'>
'Last' element grad: <class 'NoneType'>
```

Create data in tensor-format, with `requires_grad = True`

```
print("\nTensors with autograd:")
w = torch.tensor([0.43, -0.02], requires_grad = True,\
device='cpu')
print("Tensor:", w)
for wi in w:
    print(wi, "grad:", wi.grad)
```

```
Tensors with autograd:
Tensor: tensor([ 0.4300, -0.0200], requires_grad=True)

tensor(0.4300, grad_fn=<UnbindBackward0>) grad: None
tensor(-0.0200, grad_fn=<UnbindBackward0>) grad: None
```

Causing gradients to be created in the graph. The gradients are used for the gradient descent approaches, that are used for updating the weights in back propagation.
(y_a is connected with w).

```
w = torch.tensor([0.1], requires_grad=True)
y_a = 3*w
l = y_a - 2

print("w:", w, "grad:", w.grad)
print("y_a: ", y_a, "grad:", y_a.grad)

l.backward()

print("w:", w, "grad:", w.grad)
print("y_a: ", y_a, "grad:", y_a.grad)
```

```
w: tensor([0.1000], requires_grad=True) grad: None
y_a:  tensor([0.3000], grad_fn=<MulBackward0>) grad: None

w: tensor([0.1000], requires_grad=True) grad: tensor([3.])
y_a:  tensor([0.3000], grad_fn=<MulBackward0>) grad: None
```

4. Tensor operations

Sum

```
 # hunger_level, strength, cleverness, evilness
trollas_x = [0.1, 0.7, 0.05, 0.9]
bilbos_x = [0.9, 0.1, 0.9, 0.05]
evil_wizard_x = [0.1, 0.9, 0.9, 0.9]
creatures_data = torch.tensor([trollas_x, bilbos_x,\
    evil_wizard_x])

print(creatures_data)
print("Sum:", torch.sum(creatures_data))
print("Sum, dim=0:", torch.sum(creatures_data, dim=0))
print("Sum, dim=1:", torch.sum(creatures_data, dim=1))
```

```
tensor([[0.1000, 0.7000, 0.0500, 0.9000],
        [0.9000, 0.1000, 0.9000, 0.0500],
        [0.1000, 0.9000, 0.9000, 0.9000]])
Sum: tensor(6.5000)
Sum, dim=0: tensor([1.1000, 1.7000, 1.8500, 1.8500])
Sum, dim=1: tensor([1.7500, 1.9500, 2.8000])
```

Multiplication

(where one tensor has `requires_grad=True`)

```
# hunger_level, strength, cleverness, evilness
trollas_x = torch.tensor([0.1, 0.7, 0.05, 0.9])
dangerous_weights = torch.tensor([0.7, 0.9, 0.2, 0.8],\
    requires_grad=True)

product = dangerous_weights * trollas_x
print(product)
```

```
tensor([0.0700, 0.6300, 0.0100, 0.7200],
grad_fn=<MulBackward0>)
```

Dot product

With two vectors (the dangerousness level of Trolla):

```
# hunger_level, strength, cleverness, evilness
trollas_x = torch.tensor([0.1, 0.7, 0.05, 0.9])
dangerous_weights = torch.tensor([0.7, 0.9, 0.2, 0.8], \
    requires_grad=True)

dangerous_level = dangerous_weights @ trollas_x
print("Dangerous level:", dangerous_level)

print("Count it manually:", \
    0.1*0.7 + 0.7*0.9 + 0.05*0.2 + 0.9*0.8)
```

```
Dangerous level: tensor(1.4300, grad_fn=<DotBackward0>)
Count it manually: 1.4300000000000002
```

Dot product

With a matrix & a vector (all three creatures):

```
# hunger_level, strength, cleverness, evilness
trollas_x = [0.1, 0.7, 0.05, 0.9]
bilbos_x = [0.9, 0.1, 0.9, 0.05]
evil_wizard_x = [0.1, 0.9, 0.9, 0.9]
creatures_data = torch.tensor([trollas_x, bilbos_x,\
    evil_wizard_x])
dangerous_weights = torch.tensor([0.7, 0.9, 0.2, 0.8], \
    requires_grad=True)

dangerous_levels = creatures_data @ dangerous_weights
print("Dangerous levels:", dangerous_levels)

print(0.1*0.7 + 0.7*0.9 + 0.05*0.2 + 0.9*0.8, \
    0.9*0.7 + 0.1*0.9 + 0.9*0.2 + 0.05*0.8, \
    0.1*0.7 + 0.9*0.9 + 0.9*0.2 + 0.9*0.8)
```

```
Dangerous levels: tensor([1.4300, 0.9400, 1.7800],
    grad_fn=<MvBackward0>)
1.4300000000000002 0.9400000000000001 1.7800000000000002
```

5. Model parameters

Model parameters automatically get `requires_grad = True`

```
print("\nParameter tensors:")
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

parameters = model.parameters()
for param in list(parameters):
    print("Parameters: ", param, "Grad:", param.grad)
```

```
Parameter tensors:
Parameters:  Parameter containing:
tensor([[ 0.4693, -0.6568]], requires_grad=True)
Grad: None
```

The model parameters get random values to start with, so if you run the code again, the tensor will have different values.

```
print("\nParameter tensors:")
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

parameters = model.parameters()
for param in list(parameters):
    print("Parameters: ", param, "Grad:", param.grad)
```

```
Parameter tensors:
Parameters:  Parameter containing:
tensor([[ 0.6407, -0.6389]], requires_grad=True)
Grad: None
```

A linear model with three input features, will have three parameters.

```
print("\nParameter tensors:")
model = torch.nn.Linear(3, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

parameters = model.parameters()
for param in list(parameters):
    print("Parameters: ", param, "Grad:", param.grad)
```

```
Parameter tensors:
Parameters:  Parameter containing:
tensor([[ 0.3026,  0.5425, -0.4924]], requires_grad=True)
Grad: None
```

6. Tensors during model training

```
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for param in list(model.parameters()):
    print("Before training: ", param, "Grad:", param.grad)
for epoch in range(3):
    for example in data:
        x = example[:2]
        y = example[2:]

        model.zero_grad()
        y_pred = model(x)
        loss = (y_pred - y)**2
        loss.backward()
        optimizer.step()

        for param in list(model.parameters()):
            print("Parameters:", param, "Grad:", param.grad)
```

```
Before training:  Parameter containing:
tensor([[-0.5267,  0.3705]], requires_grad=True) Grad: None
Parameters: Parameter containing:
tensor([[0.1029, 0.4404]], requires_grad=True)
Grad: tensor([[-6295.9390,  -699.5488]])
```

Use gradient content so see how the parameters should be updated (lr = learning rate):

$w_{new} = w_{old} - lr * grad$

Print-out from code (No gradients before training starts. Training creates gradients, which are used for updating the two parameters):

```
Before training with first sample:  Parameter containing:
tensor([[-0.5267,  0.3705]], requires_grad=True) Grad: None

Updated weights after first sample: Parameter containing:
tensor([[0.1029, 0.4404]], requires_grad=True)
Grad: tensor([[-6295.9390,  -699.5488]])
```

```
1 lr=0.0001
2 new_w_0 = -0.5267 - lr*-6295.9390
3 new_w_1 = 0.3705 - lr*-699.5488
4 print(new_w_0, new_w_1)
```

```
0.1028939000000001 0.44045488
```

When are the gradients set? (and when are they reset to zero?).

When are the weights updated?

Create a print function "p_p()" to print out the what happens to our weights during the different steps in the training:

```
def p_p(model, text):
    print(text +": ")
    for param in list(model.parameters()):
        print(str(param).replace("Parameter containing:\n", ""
         "Grad:", param.grad)
```

When are the gradients: set? reset to zero? When are the weights updated?

```
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(1):
    for data_i, example in enumerate(data):
        #p_p(model,f"\n1. START DATA POINT {data_i}")
        x, y = example[:2], example[2:]
        model.zero_grad()
        #p_p(model,"2. after zero_grad")
        y_pred = model(x)
        #p_p(model,"3. after predict")
        loss = (y_pred - y)**2
        #p_p(model,"4. after loss calculation")
        loss.backward()
        #p_p(model,"5. after loss.backward()")
        optimizer.step()
        #p_p(model,"6. after optimizer.step()")
```

"5. after loss.backward()": gradients are set. "6. after optimizer.step()": weights are updated.

```
for data_i, example in enumerate(data):
    #p_p(model,f"\n1. START DATA POINT {data_i}")
    x, y = example[:2], example[2:]
    model.zero_grad() #; p_p(model,"2. after zero_grad")
    y_pred = model(x) #; p_p(model,"3. after predict")
    loss = (y_pred - y)**2 #; p_p(model,"4. after loss calcula
    loss.backward() #; p_p(model,"5. after loss.backward()")
    optimizer.step() #; p_p(model,"6. after optimizer.step()")
```

```
1. START DATA POINT 0:  tensor([[ 0.2945, -0.0957]], requires_grad=True)
    Grad: None
2. after zero_grad:  tensor([[ 0.2945, -0.0957]], requires_grad=True)
    Grad: None
3. after predict:  tensor([[ 0.2945, -0.0957]], requires_grad=True)
    Grad: None
4. after loss calculation:  tensor([[ 0.2945, -0.0957]], requires_grad=True)
    Grad: None
5. after loss.backward():  tensor([[ 0.2945, -0.0957]], requires_grad=True)
    Grad: tensor([[-188.8544,  -20.9838]])
6. after optimizer.step():  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: tensor([[-188.8544,  -20.9838]])
loss tensor([2.2465], grad_fn=<PowBackward0>)
```

"2. after zero_grad:" gradients are reset to None.

```
for data_i, example in enumerate(data):
    #p_p(model,f"\n1. START DATA POINT {data_i}")
    x, y = example[:2], example[2:]
    model.zero_grad() #; p_p(model,"2. after zero_grad")
    y_pred = model(x) #; p_p(model,"3. after predict")
    loss = (y_pred - y)**2 #; p_p(model,"4. after loss calcula
    loss.backward() #; p_p(model,"5. after loss.backward()")
    optimizer.step() #; p_p(model,"6. after optimizer.step()")
```

```
1. START DATA POINT 1:  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: tensor([[-188.8544,  -20.9838]])
2. after zero_grad:  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: None
3. after predict:  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: None
4. after loss calculation:  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: None
5. after loss.backward():  tensor([[ 0.3134, -0.0936]], requires_grad=True)
    Grad: tensor([[-80.8470, -10.2338]])
6. after optimizer.step():  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: tensor([[-80.8470, -10.2338]])
loss tensor([0.2618], grad_fn=<PowBackward0>)
```

```python
for data_i, example in enumerate(data):
    #p_p(model,f"\n1. START DATA POINT {data_i}")
    x, y = example[:2], example[2:]
    model.zero_grad() #; p_p(model,"2. after zero_grad")
    y_pred = model(x) #; p_p(model,"3. after predict")
    loss = (y_pred - y)**2 #; p_p(model,"4. after loss calcula
    loss.backward() #; p_p(model,"5. after loss.backward()")
    optimizer.step() #; p_p(model,"6. after optimizer.step()")
```

```
1. START DATA POINT 2:  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: tensor([[-80.8470, -10.2338]])
2. after zero_grad:  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: None
3. after predict:  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: None
4. after loss calculation:  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: None
5. after loss.backward():  tensor([[ 0.3214, -0.0926]], requires_grad=True)
    Grad: tensor([[110.2561,   3.3925]])
6. after optimizer.step():  tensor([[ 0.3104, -0.0930]], requires_grad=True)
    Grad: tensor([[110.2561,   3.3925]])
loss tensor([0.7193], grad_fn=<PowBackward0>)
```

Finish the training:

```
for data_i, example in enumerate(data):
    #p_p(model,f"\n1. START DATA POINT {data_i}")
    x, y = example[:2], example[2:]
    model.zero_grad() #; p_p(model,"2. after zero_grad")
    y_pred = model(x) #; p_p(model,"3. after predict")
    loss = (y_pred - y)**2 #; p_p(model,"4. after loss calcula
    loss.backward() #; p_p(model,"5. after loss.backward()")
    optimizer.step() #; p_p(model,"6. after optimizer.step()")
```

After a number of epochs, the loss gets very small.

That is, our model gets really good at predicting meters, given feet and inches. Then we can stop the training.

```
w = [  0.3047,   0.0254]
x_training: tensor([95.,  2.])
y_expected: 29.010000228881836
y_predicted: 29.000080108642578
loss: 9.840878192335367e-05
```

Using the model on unseen data.

```
data = torch.tensor(data.astype(np.float32), device='cpu')
model = torch.nn.Linear(2, 1, bias=False)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(3):
    for nr, example in enumerate(data):
        ...
print(model.weight)
unseen_feet_inch = torch.tensor(np.array([17, 4],\
    dtype=np.float32))
in_meters = model(unseen_feet_inch)
print("In meters:", in_meters)
```

```
Parameter containing:
tensor([[0.3049, 0.0254]], requires_grad=True)
In meters: tensor([5.2853], grad_fn=<SqueezeBackward4>)
```

But we don't need the graph connections, that we use to update parameters to use the model. So we can apply

`with torch.no_grad()`

```
with torch.no_grad():
    print(model.weight)
    unseen_feet_inch = torch.tensor(np.array([17, 4],\
    dtype=np.float32))
    in_meters = model(unseen_feet_inch)
    print("In meters:", in_meters)
```

Then we can calculate meters without adding the results to the graph

```
Parameter containing:
tensor([[0.3049, 0.0254]], requires_grad=True)
In meters: tensor([5.2853])
```

The model prediction is not magical at all,
we just use the weights and a dot product:

```
with torch.no_grad():
    unseen_feet_inch = torch.tensor(np.array([17, 4],\
    dtype=np.float32))
    print("unseen_feet_inch: ", unseen_feet_inch)
    for w in model.weight:
        print("w: ", w)
        in_meters_manual = w @ unseen_feet_inch
        print("In meters manual (w @ unseen_feet_inch):"\
                , in_meters_manual)

    in_meters = model(unseen_feet_inch)
    print("In meters (using model):", in_meters)
```

```
unseen_feet_inch:  tensor([17.,  4.])
w:  tensor([0.3049, 0.0254], requires_grad=True)
In meters manual (w @ unseen_feet_inch): tensor(5.2853)
In meters (using model): tensor([5.2853])
```

Or if we forgot what dot product is: 0.3049*17 + 0.0254 * 4 = 5.285

We could also learn bias. (Before, we used the knowledge that 0 feet and 0 inches equals 0 meters. So we know we don't need any bias)

```
model = torch.nn.Linear(2, 1, bias=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(3):
    for data_i, example in enumerate(data):
        x, y = example[:2], example[2:]
        model.zero_grad()
        y_pred = model(x)
        loss = (y_pred - y)**2
        loss.backward()
        optimizer.step()
        p_p(model,"")
        print("loss", loss)
```

```
tensor([[ 0.3432, -0.0816]], requires_grad=True) Grad: tensor([[1243.4634,  138
tensor([0.2630], requires_grad=True) Grad: tensor([19.7375])
loss tensor([97.3924], grad_fn=<PowBackward0>)
:
tensor([[ 0.3080, -0.0860]], requires_grad=True) Grad: tensor([[352.8797,  44.6
tensor([0.2626], requires_grad=True) Grad: tensor([4.4668])
loss tensor([4.9881], grad_fn=<PowBackward0>)
```

After many epochs, the model has learnt that the bias should be (close to) zero.

```
model = torch.nn.Linear(2, 1, bias=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(200):
    for data_i, example in enumerate(data):
        x, y = example[:2], example[2:]
        model.zero_grad()
        y_pred = model(x)
        loss = (y_pred - y)**2
        loss.backward()
        optimizer.step()
        p_p(model,"")
        print("loss", loss)
```

```
tensor([[0.3049, 0.0254]], requires_grad=True) Grad: tensor([[-1.8783, -0.0395]
tensor([0.0004], requires_grad=True) Grad: tensor([-0.0198])
loss tensor([9.7729e-05], grad_fn=<PowBackward0>)
```

7. Looking at some classes

torch.nn.Linear:

Linear

class **torch.nn.Linear(*in_features*, *out_features*, *bias*=*True*, *device*=*None*, *dtype*=*None*)** [source]

Applies an affine linear transformation to the incoming data: $y = xA^T + b$.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

Parameters:

- **in_features** (*int*) – size of each input sample
- **out_features** (*int*) – size of each output sample
- **bias** (*bool*) – If set to `False`, the layer will not learn an additive bias. Default: `True`

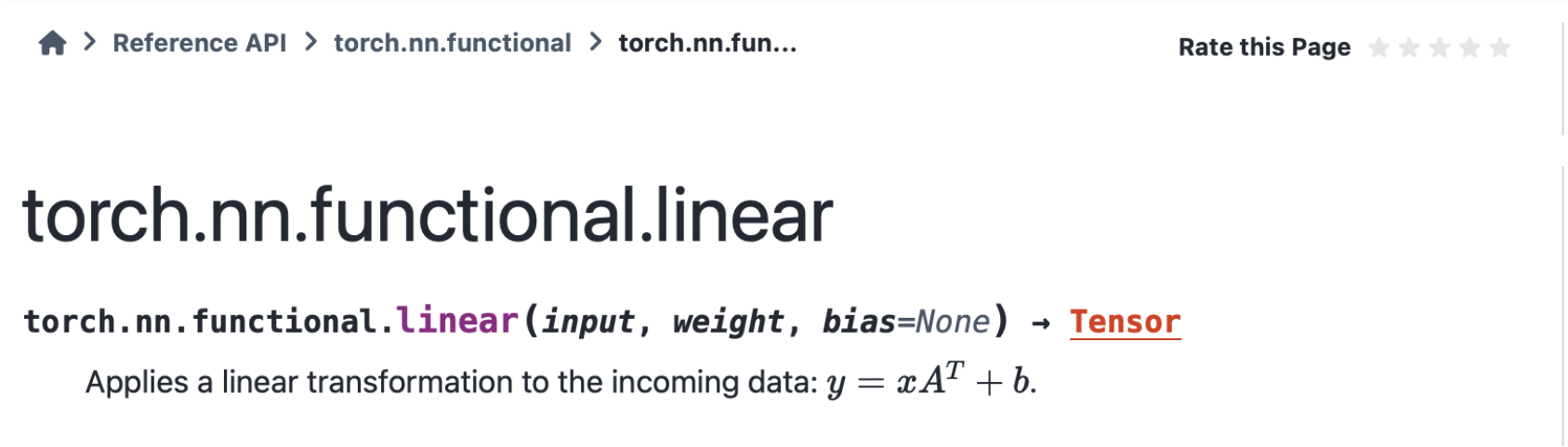
https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html

torch.nn.Linear (calls init of superclass, and then creates the parameters.)

```
class Linear(Module):
    def __init__(self, in_features: int, out_features: int,
        bias: bool = True, device=None, dtype=None) -> None:
        factory_kwargs = {"device": device, "dtype": dtype}

        super().__init__()

        self.in_features = in_features
        self.out_features = out_features
        self.weight = Parameter(
            torch.empty((out_features, in_features),\
            **factory_kwargs))
        if bias:
            self.bias = Parameter(torch.empty(out_features, \
            **factory_kwargs))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()
```

https://github.com/pytorch/pytorch/blob/v2.10.0/torch/nn/modules/linear.py#L53

```
from torch.nn import functional as F

class Linear(Module):
    def __init__(self, in_features: int, out_features: int,
        bias: bool = True, device=None, dtype=None) -> None:
        factory_kwargs = {"device": device, "dtype": dtype}
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.weight = Parameter(
            torch.empty((out_features, in_features),\
            **factory_kwargs)
        )
        if bias:
            self.bias = Parameter(torch.empty(out_features, \
            **factory_kwargs))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()

    def forward(self, input: Tensor) -> Tensor:
        return F.linear(input, self.weight, self.bias)
    ...
```



https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.linear.html

(https://docs.pytorch.org/docs/stable/nn.functional.html)

```
class Linear(Module):
    def __init__(self, in_features: int, out_features: int,
        bias: bool = True, device=None, dtype=None) -> None:
        ...

    def forward(self, input: Tensor) -> Tensor:
        return F.linear(input, self.weight, self.bias)
    ...
```

What in our code for training the model, has the affect that the `forward` method in `Linear` is called?

```
model = torch.nn.Linear(2, 1, bias=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(200):
    for data_i, example in enumerate(data):
        x, y = example[:2], example[2:]
        model.zero_grad()
        y_pred = model(x)
        loss = (y_pred - y)**2
        loss.backward()
        optimizer.step()
```

Find the superclass of Linear: Module.

```
class Linear(Module):
    def __init__(self, in_features: int, out_features: int,
        bias: bool = True, device=None, dtype=None) -> None:
        ...

    def forward(self, input: Tensor) -> Tensor:
        return F.linear(input, self.weight, self.bias)
    ...
```

***class* torch.nn.Module(**args*, ***kwargs*)** [source]

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F


class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

torch.nn.Module has defined the magic method __call__ (original typing removed in the code below)

```
class Module:
    ...

    __call__ = _wrapped_call_impl
```

https://github.com/pytorch/pytorch/blob/v2.10.0/torch/nn/modules/module.py#L407
Calling `_wrapped_call_impl`, eventually leads to the method `forward()` being called.

More complex models are normally constructed as subclasses of `torch.nn.Module`

```
import torch.nn as nn
class MyLinear(nn.Module):
    def __init__(self):
        super().__init__()
        self.model_1 = nn.Linear(2, 1)

    def forward(self, x):
        print("Calling the forward method")
        return self.model_1(x)

model = MyLinear()
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)

for epoch in range(200):
    for data_i, example in enumerate(data):
        x, y = example[:2], example[2:]
        model.zero_grad()
        y_pred = model(x)
        loss = (y_pred - y)**2
        loss.backward()
        optimizer.step()
```

```python
import torch.nn as nn
class MyLinear(nn.Module):
    def __init__(self):
        super().__init__()
        self.model_1 = nn.Linear(2, 1)

    def forward(self, x):
        print("Calling the forward method")
        return self.model_1(x)

model = MyLinear()
optimizer = torch.optim.SGD(model.parameters(), lr=0.0001)
```

```
Calling the forward method
:
tensor([[0.2615, 0.4276]], requires_grad=True) robGrad: tensor([[-318.9305,  -3
tensor([-0.5814], requires_grad=True) Grad: tensor([-5.0624])
loss tensor([6.4069], grad_fn=<PowBackward0>)
...
Calling the forward method
:
tensor([[0.3049, 0.0254]], requires_grad=True) Grad: tensor([[-1.8765, -0.0395]
tensor([0.0009], requires_grad=True) Grad: tensor([-0.0198])
loss tensor([9.7540e-05], grad_fn=<PowBackward0>)
```